



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Технології розроблення програмного забезпечення
Лабораторна робота №8
на тему «ШАБЛОНИ «COMPOSITE», «FLYWEIGHT»,
«INTERPRETER», «VISITOR».»

Виконала:
Студентка групи ІА-22
Патрик Вікторія

Перевірив:
Мягкий Михайло Юрійович

Київ 2024

Зміст

Тема	3
Короткі теоретичні відомості.....	3
Реалізація не менше 3-х класів відповідно до обраної теми	8
Реалізація шаблону за обраною темою	10
1. FontFlyweight.....	10
2. FontFlyweightFactory	11
3. FlyweightService	12
4. FlyweightController.....	13
Результати виконання	14
Посилання на код на GitHub	14
Діаграма класів для патерну «Flyweight».....	15
Переваги застосування патерну «Flyweight»	16

ШАБЛони «COMPOSITE», «FLYWEIGHT», «INTERPRETER», «VISITOR»

Мета: ознайомитися з шаблонами проектування «COMPOSITE», «FLYWEIGHT», «INTERPRETER», «VISITOR» та набутти практичних навичок їх застосування. Реалізувати частину функціоналу програми за допомогою одного з розглянутих шаблонів для досягнення конкретних функціональних можливостей та забезпечення ефективної взаємодії між класами.

Тема:

..3 Текстовий редактор (strategy, command, observer, template method, flyweight, SOA)

Текстовий редактор повинен вміти розпізнавати текстові файли в будь-якій кодуванні, мати розширені функції редагування: макроси, сніппети, підказки, закладки, перехід на рядок / сторінку, підсвічування синтаксису (для однієї мови програмування або розмітки на розсуд студента).

Хід роботи:

Короткі теоретичні відомості:

Шаблони роботи з БД при розробці корпоративних додатків: Існує набір сучасних шаблонів, які використовуються для розробників широко-масштабних корпоративних додатків - з кількістю користувачів понад 100 і великою кількістю взаємопов'язаних програм.

Такі додатки вимагають окремих підходів до організації доступу до даних, оскільки кількість даних, що зберігаються, класів і клієнтів, що звертаються, зростає.

Шаблон «Active Record»: Поєднує дані й логіку в одному об'єкті, що представляє рядок БД. Широко використовується через простоту, але складність класу зростає зі збільшенням запитів, тому логіку часто виносять окремо.

Шаблон «Table Data Gateway»: Окремий клас для взаємодії з БД для кожного класу даних. Вся логіка запитів зосереджена в ньому, що підвищує гнучкість і тестованість. Часто абстрагується в базовий клас для зменшення дублювання.

Шаблон «Data Mapping»: Перетворює об'єкти даних у формат, що приймається БД чи мережею. Маппер пов'язує властивості об'єкта з колонками таблиці, вирішуючи невідповідності типів.

Шаблон «Composite»

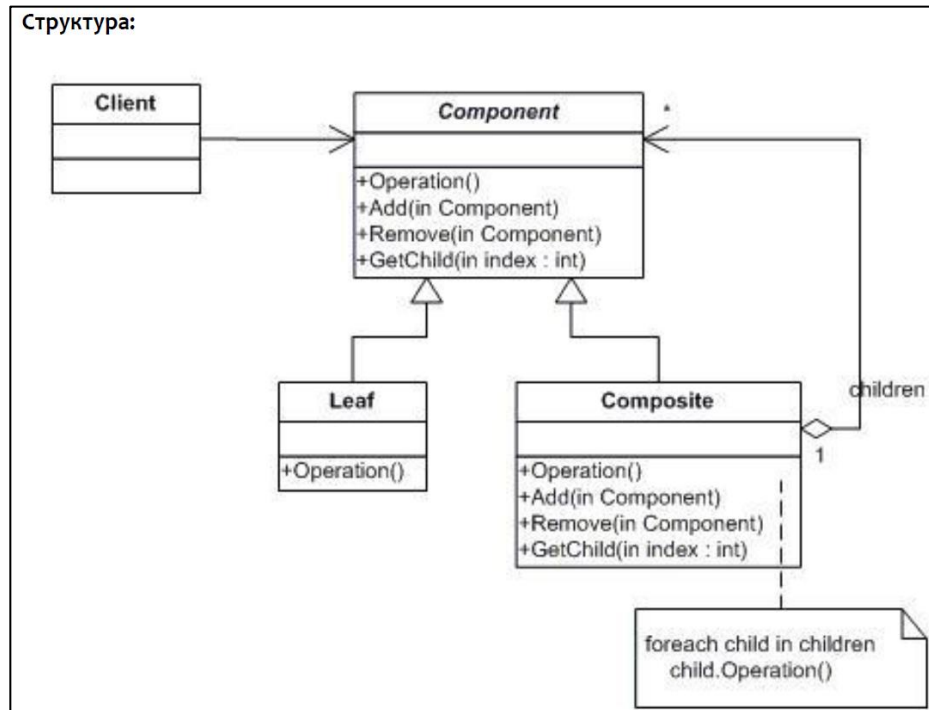


Рисунок 1. Структура шаблону «Composite»

Призначення: Патерн "Composite" використовується для представлення і роботи з деревовидними структурами, де окремі об'єкти (листи) і групи об'єктів (вузли) обробляються однаково.

Проблема: Патерн "Composite" корисний для роботи з деревовидною структурою. Наприклад, продукти й коробки, які можуть містити інші коробки. Задача: обчислити загальну ціну замовлення, не знаючи наперед структури вкладеності.

Рішення: "Composite" уніфікує роботу з продуктами й коробками через спільний інтерфейс. Продукт повертає свою ціну, коробка обчислює суму вартості свого вмісту, незалежно від рівнів вкладеності.

Приклад з життя: Армія — структура у вигляді дерева: солдати, взводи, полки, армії. Накази передаються зверху вниз за ієрархією.

Переваги:

- Спрощує роботу зі складними структурами.
- Полегшує додавання нових компонентів.
- Створює загальнодизайнерське рішення.

Недоліки:

- Може бути складним для реалізації.
- Перебір вмісту може вимагати значних ресурсів.

Шаблон «Flyweight»

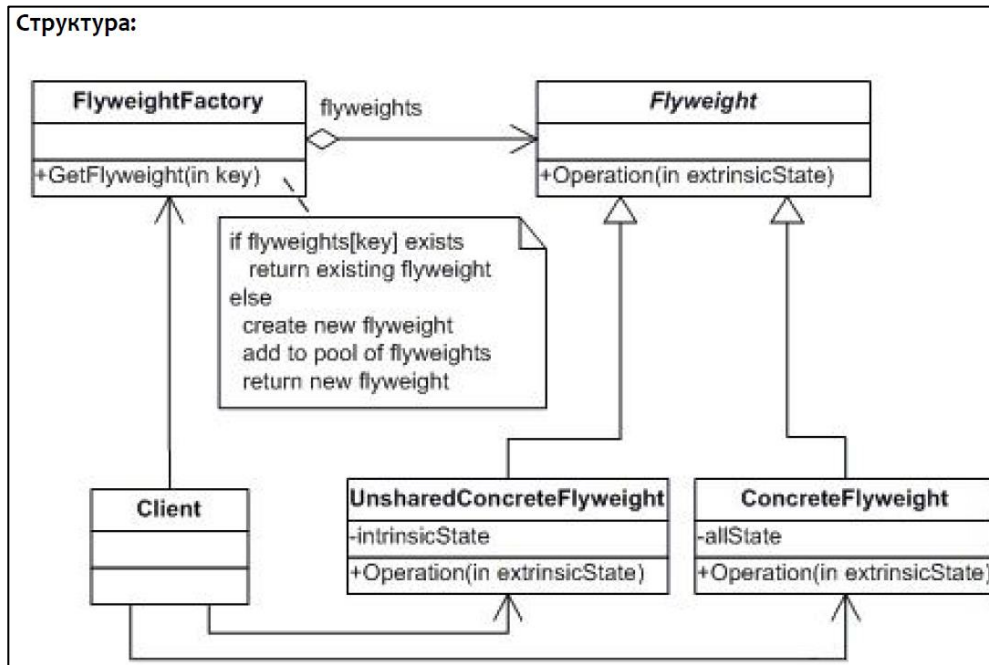


Рисунок 2. Структура шаблону «Flyweight»

Призначення: Шаблон використовується для зменшення кількості об'єктів шляхом поділу загальних даних між екземплярами. Наприклад, у графічних системах або текстових редакторах.

Проблема: В іграх або додатках із великою кількістю однакових об'єктів (наприклад, кулі, частинки, текстові символи) обсяг оперативної пам'яті стає критичним. Кожен об'єкт містить багато даних, які можуть дублюватися.

Рішення: Розділяти стан об'єктів на:

- Внутрішній – загальні дані, які зберігаються в спільному об'єкті.
- Зовнішній – унікальні дані, пов'язані з конкретним контекстом використання.

Це дозволяє значно скоротити використання пам'яті.

Приклад з життя: У ветеринарній базі даних коти мають загальні (назва породи, текстура) і унікальні (ім'я, вік) дані. Загальні дані зберігаються окремо і використовуються спільно.

Переваги:

- Економія оперативної пам'яті.
- Уніфікація структури даних.

Недоліки:

- Зростає складність коду.
- Витрачається час на пошук спільних даних.

Шаблон «Interpreter»

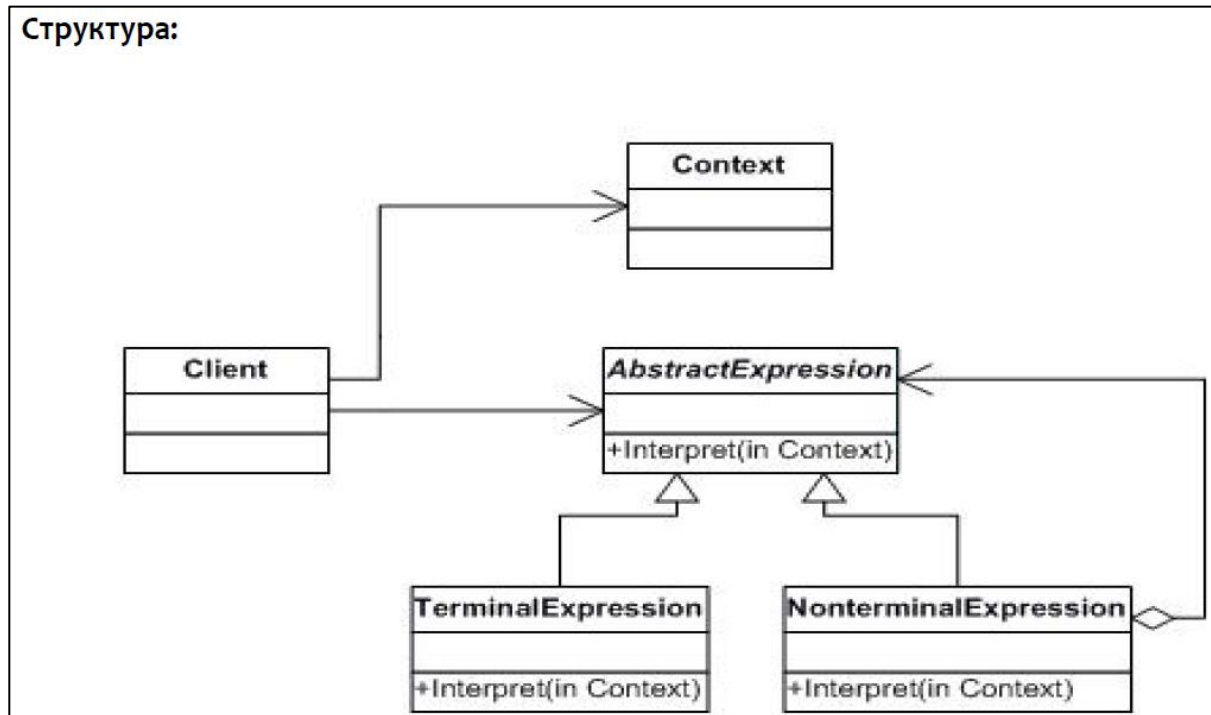


Рисунок 3. Структура шаблону «Interpreter»

Призначення: Шаблон використовується для подання граматики і створення інтерпретатора для мови (наприклад, скриптової), що дозволяє працювати з термінальними та нетермінальними символами у вигляді деревовидної структури.

Проблема: Необхідно автоматизувати розбір виразів або завдань, які часто змінюються, і реалізувати зрозумілу систему обчислення.

Рішення: Створюється абстрактне синтаксичне дерево, вузли якого представлені класами:

- Термінальний вираз – відповідає за базу рекурсії.
- Нетермінальний вираз – реалізує обробку підвиразів (рекурсивно).

Кожен вузол виконує операцію розбору з урахуванням переданого контексту.

Приклад з життя: Припустимо потрібний автоматичний розбір поставленого завдання таким чином, що воно розбивається на прості та стає зрозумілим системі.

Переваги:

- Легка розширюваність граматики.
- Простота змін у способі обчислення виразів.

Недоліки:

- Ускладнення підтримки граматики з великою кількістю правил.

Шаблон «Visitor»

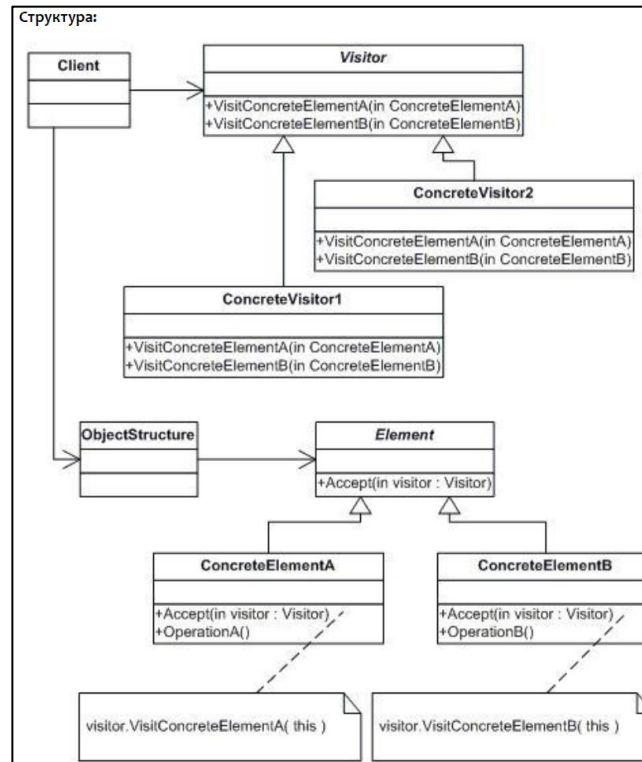


Рисунок 4. Структура шаблону «Visitor»

Призначення патерну: Патерн дозволяє вказувати операції над елементами без зміни їхньої структури. Групує однотипні операції, застосовувані до різних об'єктів.

Проблема: Уявіть програму з графами (вузли — міста, локації тощо). Завдання — реалізувати експорт графа в XML без змін у стабільному коді вузлів, але при цьому дозволити додавання нових операцій.

Рішення: Нову поведінку реалізують через окремий клас "Відвідувач". Об'єкти передають себе до методу відвідувача для виконання операцій. Це дозволяє додавати нові операції без змін у класах вузлів.

Приклад з життя: Компілятор, що генерує код для різних елементів синтаксису (виклики методів, умовні вирази). Кожен елемент передає себе відвідувачу, який виконує необхідні операції для генерації або перевірки коду.

Переваги:

- Додає нові операції без змін у коді об'єктів.
- Групує логіку, що застосовується до різних об'єктів.

Недоліки:

- Додає складність у реалізації.
- Необхідно підтримувати багато відвідувачів для різних операцій.

Реалізація не менше 3-х класів відповідно до обраної теми:

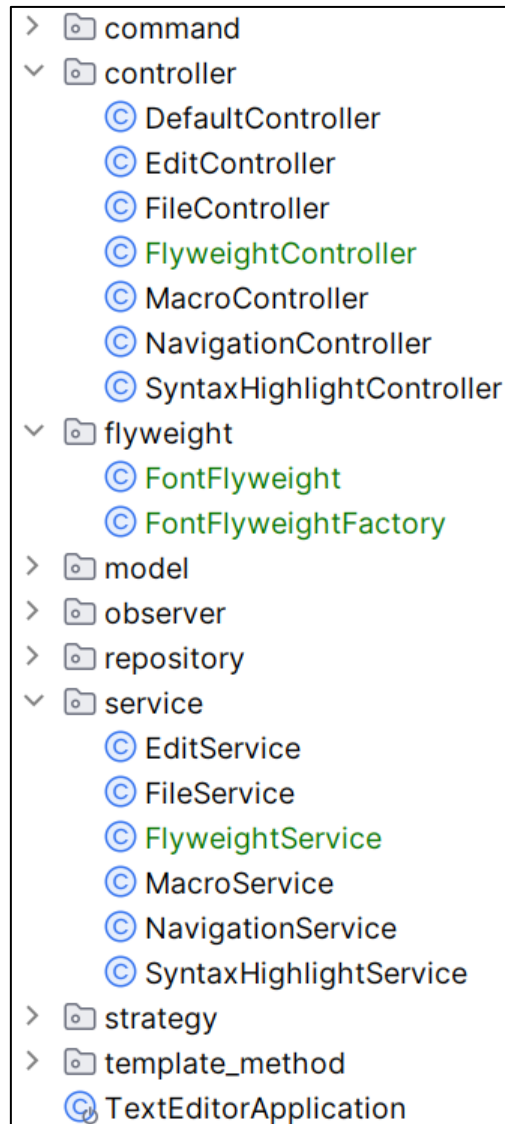


Рисунок 5. Структура проекту з використанням патерну «Flyweight».

У ході лабораторної роботи була реалізована система текстового редактора з використанням патерну Flyweight для оптимізації збереження пам'яті при роботі з повторюваними даними. Основним завданням було впровадження Flyweight для збереження даних про шрифти тексту, що дозволяє значно скоротити обсяг використання пам'яті. Структура проекту наведена на Рисунку 1. У результаті виконання роботи були реалізовані наступні кроки:

1. Розроблено структуру проекту:

- Проект поділено на логічні пакети: controller, flyweight, service, model, repository, strategy та інші, що забезпечує зручність масштабування та підтримки коду.

2. Реалізовано патерн «Flyweight»:

- Створено клас FontFlyweight, який визначає спільний внутрішній стан (назву шрифту та його розмір) для повторюваних об'єктів тексту.

- Реалізовано фабрику Flyweight:
 - Клас FontFlyweightFactory управляє створенням і кешуванням об'єктів Flyweight.
 - Використовується структура Map, яка дозволяє перевіряти, чи об'єкт із такими ж параметрами вже існує, і в разі відсутності створювати новий.
 - Створено сервіс для роботи з Flyweight:
 - Клас FlyweightService забезпечує логіку обробки тексту з використанням об'єктів Flyweight.
 - Додається метод для виводу тексту з інформацією про шрифт, а також метод для підрахунку кількості створених Flyweight-об'єктів.
- 3. Реалізовано сервісний рівень:**
- Створено сервіс FlyweightService, який:
 - Забезпечує обробку тексту через Flyweight-об'єкти.
 - Використовує фабрику FontFlyweightFactory для оптимізації створення шрифтів.
 - Підтримує відображення тексту з деталями (символ, шрифт, розмір).
- 4. Створено контролери для роботи з Flyweight:**
- FlyweightController:
 - Реалізує REST API для роботи з функціоналом Flyweight.
 - Підтримує два основних ендпойнти:
 - POST /flyweight/display — для відображення тексту з певним шрифтом.
 - GET /flyweight/count — для отримання кількості створених Flyweight-об'єктів.
- 5. Перевірено функціонал системи:**
- Тестування API через Postman підтвердило коректність створення та використання Flyweight-об'єктів.
 - Проведено перевірку кешування: система повторно використовує вже існуючі Flyweight-об'єкти для однакових шрифтів, що значно зменшує кількість створених об'єктів і споживання пам'яті.
 - Тестування включало перевірку з різними параметрами тексту, шрифтів і розмірів.

Цей підхід забезпечує ефективне використання пам'яті та спрощує організацію коду. Завдяки впровадженню Flyweight текстовий редактор став здатним обробляти великі обсяги даних із мінімальними витратами пам'яті. Система легко масштабується, дозволяючи додавати нові елементи для оптимізації (наприклад, стилі або кольори тексту) без змін у наявному коді.

Реалізація шаблону за обраною темою:

1. FontFlyweight

```
1 package com.example.text_editor.flyweight;
2
3 5 usages new *
4 public class FontFlyweight {
5     2 usages
6     private final String fontName;
7     2 usages
8     private final int fontSize;
9
10    1 usage new *
11    public FontFlyweight(String fontName, int fontSize) {
12        this.fontName = fontName;
13        this.fontSize = fontSize;
14    }
15
16    1 usage new *
17    public String getFontName() {
18        return fontName;
19    }
20
21    1 usage new *
22    public int getFontSize() {
23        return fontSize;
24    }
25 }
```

Рисунок 6. Клас FontFlyweight.

Цей клас, показаний на Рисунку 6, реалізує базову структуру для збереження спільного стану шрифтів, таких як назва (fontName) і розмір (fontSize). Він є ядром патерну Flyweight і використовується для створення об'єктів, які можуть повторно використовуватися. Конструктор ініціалізує значення, а геттери дозволяють доступ до цих параметрів.

2. FontFlyweightFactory

```
1 package com.example.text_editor.flyweight;
2
3 import java.util.HashMap;
4 import java.util.Map;
5
6 3 usages new *
7 public class FontFlyweightFactory {
8     4 usages
9     private static final Map<String, FontFlyweight> flyweights = new HashMap<>();
10
11     1 usage new *
12     public static FontFlyweight getFont(String fontName, int fontSize) {
13         String key = fontName + fontSize;
14         if (!flyweights.containsKey(key)) {
15             flyweights.put(key, new FontFlyweight(fontName, fontSize));
16         }
17         return flyweights.get(key);
18     }
19
20     1 usage new *
21     public static int getFlyweightCount() {
22         return flyweights.size();
23     }
24 }
```

Рисунок 7. Клас FontFlyweightFactory.

Цей клас, показаний на Рисунок 7, є фабрикою для керування Flyweight-об'єктами. Він зберігає об'єкти FontFlyweight у мапі, забезпечуючи їх повторне використання, якщо запитуються ті ж самі властивості шрифту. Метод getFont перевіряє існування об'єкта, створює новий лише за потреби та повертає відповідний Flyweight. Додатково метод getFlyweightCount дозволяє отримати кількість створених об'єктів.

3. FlyweightService

```
1 package com.example.text_editor.service;
2
3 import com.example.text_editor.flyweight.FontFlyweight;
4 import com.example.text_editor.flyweight.FontFlyweightFactory;
5 import org.springframework.stereotype.Service;
6
7 3 usages new *
8 @Service
9 public class FlyweightService {
10     1 usage new *
11     public void displayText(String text, String fontName, int fontSize) {
12         FontFlyweight font = FontFlyweightFactory.getFont(fontName, fontSize);
13         for (char character : text.toCharArray()) {
14             System.out.println("Character: " + character + ", Font: " + font.getFontName() + ", Size: " + font.getFontSize());
15         }
16     }
17
18     1 usage new *
19     public int getFlyweightCount() {
20         return FontFlyweightFactory.getFlyweightCount();
21     }
22 }
```

Рисунок 8. Клас FlyweightService.

Цей сервіс, показаний на Рисунку 8, реалізує основну логіку роботи з Flyweight-об'єктами. Метод `displayText` використовує фабрику для отримання потрібного шрифту та демонструє текст із відповідними властивостями шрифту. Він також виводить інформацію про кожен символ тексту, включаючи його шрифт та розмір. Метод `getFlyweightCount` забезпечує взаємодію з фабрикою для отримання кількості Flyweight-об'єктів.

4. FlyweightController

```
1 package com.example.text_editor.controller;
2
3 import org.springframework.http.ResponseEntity;
4 import org.springframework.web.bind.annotation.*;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import com.example.text_editor.service.FlyweightService;
7
8 new *
9 @RestController
10 @RequestMapping("/flyweight")
11 public class FlyweightController {
12
13     3 usages
14     private final FlyweightService flyweightService;
15
16     new *
17     @Autowired
18     public FlyweightController(FlyweightService flyweightService) {
19         this.flyweightService = flyweightService;
20     }
21 }
```

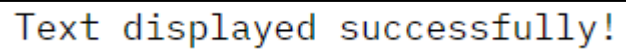
Рисунок 9.1. Клас FlyweightController.

```
19 new *
20 @PostMapping("/display")
21 public ResponseEntity<String> displayText(
22     @RequestParam(required = false) String text,
23     @RequestParam(required = false) String fontName,
24     @RequestParam(required = false) Integer fontSize) {
25     if (text == null || text.isEmpty()) {
26         return ResponseEntity.badRequest().body("Error: 'text' parameter is missing or empty.");
27     }
28     if (fontName == null || fontName.isEmpty()) {
29         return ResponseEntity.badRequest().body("Error: 'fontName' parameter is missing or empty.");
30     }
31     if (fontSize == null || fontSize <= 0) {
32         return ResponseEntity.badRequest().body("Error: 'fontSize' parameter is missing or invalid.");
33     }
34
35     flyweightService.displayText(text, fontName, fontSize);
36     return ResponseEntity.ok().body("Text displayed successfully!");
37 }
38
39 new *
40 @GetMapping("/count")
41 public String getFlyweightCount() {
42     return "Number of flyweights created: " + flyweightService.getFlyweightCount();
43 }
```

Рисунок 9.2. Клас FlyweightController.

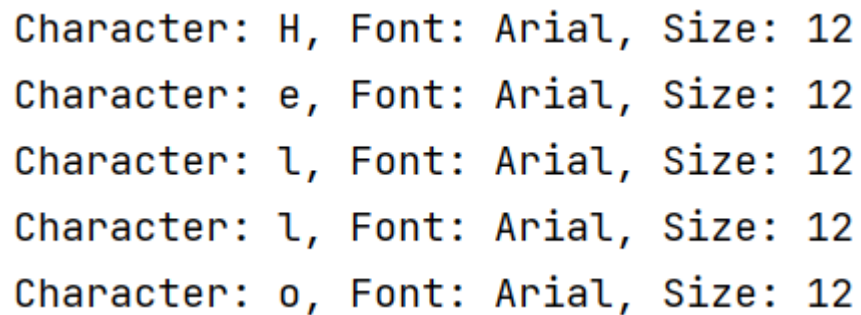
Цей контролер, показаний на Рисунках 9.1 та 9.2, надає API для взаємодії з системою Flyweight. Метод displayText дозволяє відправляти текст із параметрами шрифту через HTTP-запити, викликаючи відповідні методи сервісу. Він також обробляє помилки, якщо запити містять некоректні параметри. Метод getFlyweightCount дозволяє клієнту отримати кількість створених Flyweight-об'єктів через API.

Результати виконання:



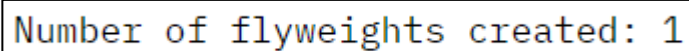
```
Text displayed successfully!
```

Рисунок 10.1. Результат виконання роботи.



```
Character: H, Font: Arial, Size: 12  
Character: e, Font: Arial, Size: 12  
Character: l, Font: Arial, Size: 12  
Character: l, Font: Arial, Size: 12  
Character: o, Font: Arial, Size: 12
```

Рисунок 10.2. Результат виконання роботи.



```
Number of flyweights created: 1
```

Рисунок 10.3. Результат виконання роботи.

Посилання на код на GitHub: https://github.com/vvvvvvvvvviktoriiia/text_editor

Діаграма класів для патерну «Flyweight»:

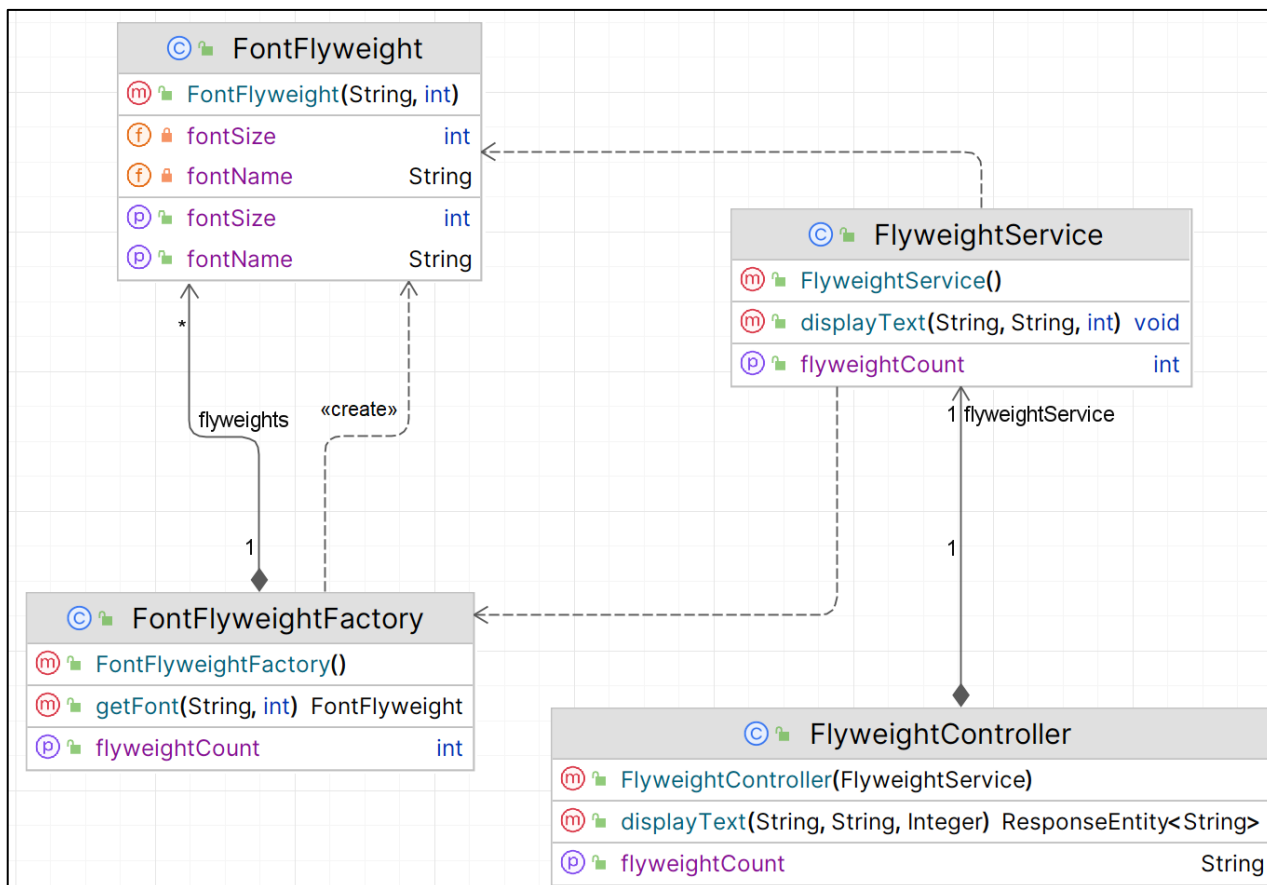


Рисунок 11. Діаграма класів для патерну «Flyweight».

Раніше об'єкти текстового редактора, такі як шрифти чи стилі, могли створюватися для кожного текстового фрагмента окремо, що призводило до:

- Високого споживання пам'яті: Кожен об'єкт дублювався, навіть якщо його властивості (наприклад, шрифт і розмір) були однаковими.
- Складності оптимізації: Створення великої кількості об'єктів ускладнювало управління пам'яттю.
- Дублювання об'єктів: Ідентичні шрифти створювалися багаторазово для різних фрагментів тексту.

Патерн "Flyweight" вирішив цю проблему, дозволивши:

- Повторне використання спільного стану через централізовану фабрику об'єктів `FontFlyweightFactory`.
- Оптимізацію пам'яті: Внутрішній стан шрифтів (назва, розмір) зберігається у `Flyweight`-об'єктах, які повторно використовуються.
- Зменшення дублювання: Однакові шрифти створюються лише один раз.

Переваги застосування патерну «Flyweight»:

1. Ефективне використання пам'яті:
 - Усі ідентичні шрифти кешуються у фабриці, що значно знижує кількість створених об'єктів.
2. Централізоване управління:
 - Фабрика Flyweight (FontFlyweightFactory) забезпечує створення й управління об'єктами, гарантуючи їх унікальність.
3. Гнучкість:
 - Легко додати нові властивості шрифтів, не змінюючи існуючий код.
4. Масштабованість:
 - Flyweight підтримує роботу з великим обсягом тексту, мінімізуючи вплив на пам'ять.
5. Модульність:
 - Розділення логіки між фабрикою (FontFlyweightFactory), сервісом (FlyweightService) і контролером (FlyweightController) забезпечує зручність у підтримці та розширенні системи.

Висновок: У даній лабораторній роботі реалізовано патерн "Flyweight", який дозволив оптимізувати управління пам'яттю у текстовому редакторі. Завдяки цій реалізації логіка роботи зі шрифтами стала уніфікованою, легко масштабованою та оптимізованою. Кількість об'єктів у системі зведена до мінімуму, що забезпечило ефективне використання ресурсів.

Крім того, додавання нових типів шрифтів або їх властивостей тепер можливе без змін у базовій логіці системи. Це зробило текстовий редактор не лише більш продуктивним з точки зору використання пам'яті, але й зручним для подальшого розвитку та підтримки. Реалізація патерну "Flyweight" дозволила стандартизувати управління шрифтами, забезпечивши гнучкість, масштабованість та оптимізацію ресурсів у системі.